

3 Debugging

As soon as a software project exceeds a certain size, errors creep into the source code. The aim of debugging is to find and rectify these errors. The errors usually occur in the course of development, and users often don't point them out until later. However, we have to solve the problem, and if we can solve it faster by using AI, then all the better.

In software development, we have different tools to track down problems. Temporary `print` statements (or `console.log` for web applications) in the code are the simplest and often quickest way to success. Structured logging with different categories and other metadata are helpful in applications that are used in production by customers. A debugger that is able to analyze and stop the code at runtime is an indispensable tool for complex applications.

These tools help you recognize and understand errors, but they aren't magic remedies. It's still up to you to think through the problem, rule out sources of error, and investigate the cause. Often, you can get so engrossed in your own code that you don't even notice anything that doesn't make sense. Once you're stuck in a dead end, it's difficult to find a way out on your own.

This is why modern software development often involves teamwork: in *pair programming*, you work together on the code, because four eyes (and two brains) simply recognize problems more than twice as quickly. If no colleague is available, *rubber duck debugging* often helps: grab a rubber duck and explain the problem to it. Because the duck isn't a programming professional, these explanations have to be slow and in small steps. The solution is usually obvious once you've broken down the error in this way.

But what if the rubber duck wasn't just a passive listener, but could contribute with analyses, proposed solutions, and ideas of its own? This would give you a partner in pair programming who is never grumpy or bored, al-

ways has time, and also has a good basic knowledge of almost every language and framework.

In this chapter, we'll try to eliminate errors in our own source code with AI support. Thanks to our experience in fixing the errors, we can estimate how much time savings is realized using AI.

The disadvantage of this is that our knowledge of the solution path can affect the question. This is precisely the central point in the formulation of the prompt: if you already give your AI helper a direction and send it on a certain path using keywords, it will most likely pick up on your train of thought and replicate it. If this doesn't solve the problem, it may make sense to start a new session and formulate the request in a more open and general way. This contradicts the advice we usually give in the book, namely that the prompts should be formulated as concretely as possible and with a lot of context so that the AI tool knows what it has to do.

However, if you're not sure yourself where the problem is and what the right solution is, it's better to give the AI tool a little more leeway. Ask it for different approaches and perspectives on the problem. Then, of course, it's up to you to choose the path that solves the problem between the various ideas. AI can certainly help you debug simple syntax problems, but when it gets more complex, the bottom line applies all the more: always check all answers and keep them in mind. Just because your AI helper proudly says that it has solved a problem doesn't mean that everything is working as it should.

Nevertheless, we can attest that our experience with AI-supported debugging has been very positive. The language models are good at finding and correcting deviations from the norm. If the error in your program can be described in this way, there is a high probability that your AI helper will find it.

3.1 Web Applications

The market share of web applications continues to grow unabated. The COVID-19 pandemic has made the call for *more online* applications even louder, and anyone bringing an application online today would do well not to forget the web browser as a possible frontend.

As an operating system-independent platform, the browser is developing rapidly and offers increasingly sophisticated tools for debugging web applications (keyword: *developer tools*). We want to see whether and how AI support can lead us to even faster solutions.

3.1.1 CSS Fixes

Anyone who has ever tried to create a supposedly simple layout in HTML will be familiar with the following problem: an element simply doesn't want to be in the position on the page where you would like it to be. As an example, we've used a very simple page closure as a footer element. The content is supposed to be divided into three areas: left, center, and right. Our first design has the disadvantage that the areas with Cascading Style Sheets (CSS) property `float` don't adhere to the height of the surrounding element, but can also flow beyond it.

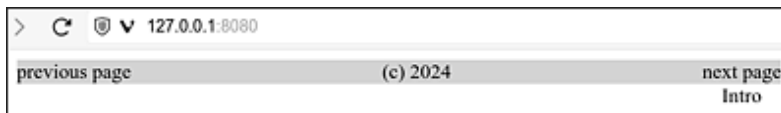


Figure 3.1 Incorrect Layout for the Footer of an HTML Page

Let's see if *Claude 3.5 Sonnet* can offer a better solution here:

Prompt: How can I make my footer elements all the same height:

```
<footer style="margin:auto;text-align:center;width:80%;">
  <span style="float:left;">previous page</span>
  <span>(c) 2024</span>
  <span style="float:right;">next page<br />Intro</span>
</footer>
```

To make the footer elements all the same height, you can use flexbox. Here's a modified version of your code that should achieve this:

```
<footer style="margin:auto; width:80%; display:flex;
  justify-content:space-between; align-items:stretch;">
  <span style="flex:1;padding:10px;">previous
page</span>
```

```

    <span style="flex:1;padding:10px;text-align:center;">
      (c) 2024</span>
    <span style="flex:1;padding:10px;text-
align:right;">next page
      <br />Intro</span>
</footer>

```

Key changes:

1. Removed float properties and used flexbox instead.
2. Added `display:flex` to the footer.
3. Used `justify-content:space-between` to distribute the spans evenly.
4. Used `align-items:stretch` to make all items stretch to the height of the tallest item. [more details]

The solution uses the powerful CSS Flexible Box Layout (Flexbox) model to position the elements correctly. Flexbox has been implemented in popular browsers for almost 10 years and can do much more than just divide an area into three equally wide columns. However, this can also make Flexbox very complex, which is why we want to ask the AI tool whether we can also use the more modern (and often simpler) grid layout.

Prompt: Would CSS `display:grid` be an option?

Yes, CSS Grid is definitely a good option for this layout. Grid can provide even more control and flexibility than Flexbox in many cases. Here's how you could implement the same layout using CSS Grid:

```

<footer style="margin:auto; width:80%; display:grid;
  grid-template-columns: 1fr 1fr 1fr;">
  <span style="padding:10px;">previous page</span>
  <span style="padding:10px;text-align:center;">(c)
2024</span>
  <span style="padding:10px;text-align:right;">next
page<br>
  Intro</span>
</footer>

```

Key points about this Grid implementation: [more details]

The result is even easier to read, and the display is identical to the Flexbox version.

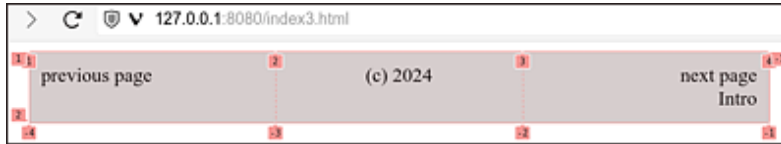


Figure 3.2 Grid Layout of an HTML Page Footer with Three Columns (with Active Developer Tools to Highlight the Three Areas)

For the first debugging task, the AI tool provided a very good solution to our problem. Even the variant we requested with a different CSS layout model remains clear and works straight away.

3.1.2 iCal Download in the Browser

Our next problem deals with the PHP programming language and the options of starting external programs from the browser. In our application, we've created an export for appointments in the standardized *iCal* format. However, clicking on the link doesn't start the calendar application as we wanted, but the text is displayed in the browser in a way that is difficult to read. We ask Claude 3.5 Sonnet if there is a quick solution:

Prompt: I have this simple PHP class that sends calendar entries over HTTPS. Unfortunately, it opens in my browser. How can I make it open in my calendar app?

To make the calendar entry open in your calendar app instead of the browser, you need to adjust the HTTP headers you're sending. Here's how you can modify your PHP class to achieve this:

[code response]

The key changes are in the `send()` method:

1. We set the `Content-Type` header to `text/calendar; charset=utf-8`. This tells the client that we're sending a calendar file.
2. We set the `Content-Disposition` header to `attachment; filename="event.ics"`. This tells the browser to treat the file as an attachment rather than trying to display it, and suggests a filename for the file.

The solution is correct, and the two changes, which are explained in detail as *key changes* in the answer, do indeed lead to the desired behavior. However, a mishap occurred during this attempt because we actually wanted to add the corresponding code passage to the prompt, so it was surprising when the AI-generated code integrated perfectly into our code base. For more complex changes, however, using your own code can certainly save further work.

3.1.3 Backend Frontend without a Proxy Server

If you use separate frontend and backend applications in web development, you'll probably be familiar with the following problem: the development server for the backend application runs on port 8000, and the bundler with live reload in the browser (e.g., Vite or Webpack) runs on port 8080. Asynchronous access from the frontend (Ajax) is blocked by the browser.

In such cases, the error messages in the browser console are detailed but difficult to understand if you haven't yet dealt with this issue. We've shortened the message in the following prompt that we sent to Claude:

Prompt: My backend application is a Golang JSON API with GIN-GONIC running on port 8000. Frontend is running with vite.js on port 8080. When I call backend routes, I get the following error:

```
Access to fetch at 'http://localhost:8000/users/me'
from origin 'http://localhost:8080' has been blocked by CORS
policy: Response to preflight request ...
```

What can I do?

To resolve this CORS (cross-origin resource sharing) issue, you need to configure your backend to allow requests from your frontend origin. Here's what you can do:

1. Install the CORS middleware for Gin:

[code response]

The AI tool recognized the problem immediately and solved it correctly. We need to include the `cors` module from the `gin-gonic` library we're using and

configure it accordingly. We're also informed that the detailed configuration can be bypassed for development purposes if the `config.AllowAllOrigins = true` option gets set.

The problem seems to be solved—only the login still doesn't work. The request with user name and password is confirmed as valid with the HTTP status code 200, but we're getting constantly redirected back to the login page. Perhaps AI can help here too?

Prompt: Great! My requests are working now, but my login functionality is not. Session variables in Go aren't present. What is the problem?

I'm glad the CORS issue is resolved! Now, regarding your login functionality and session variables not being present, there could be a few reasons for this. Let's explore some common issues and solutions:

[more details]

The AI is obviously happy that we were able to solve the first problem. We're also pleased with the follow-up answer informing us that we need to make an additional setting at the frontend and backend when transferring access data. The AI tool couldn't have known this in advance, as we didn't say anything about a login issue when we first asked. The detailed answer also contains instructions on how to save the cookies used for the session correctly (so that they also apply to other ports).

The only thing we would have expected here is that the AI tool would immediately provide a suggested solution for the library we use (`gin-gonic`) and the session management there. The answer only explained the setting for cookies in general in the Go programming language. So, we have to ask again:

Prompt: I am using `gin-gonic` sessions, how do I set relaxed restrictions for `SameSite` here?

Now we get the correct answer, and the application works as desired. In the development process, a proxy server is often used for this purpose, which sends the frontend and backend to the browser from the same port and from the same host. If you want to run the application on a production system in a

distributed manner (i.e., not on the same host), you'll have to deal with CORS anyway.

There is plenty of information about CORS available on the internet, and it's easy to find. A web search for the terms *CORS* (for *cross-origin resource sharing*) and *MDN* (for *Mozilla Developer Network*) leads us to excellent websites that describe the problem in detail. However, we don't get the finished solution for our program in the *Go* programming language, as provided by the AI tool. Of course, one could argue that this is a security-relevant topic and that code shouldn't be adopted without in-depth knowledge of the subject matter. In this case, however, AI provided good explanations, and we were able to develop the application in a very short time without spending half a day looking for solutions at Stack Overflow, which is what we did without AI help.

3.1.4 JavaScript Fetch with the Wrong Content Type

Our next problem stems from a change in which we replaced an old JavaScript library for executing asynchronous JavaScript (Ajax) web requests in the browser with the modern `fetch` interface. Most of the calls went smoothly, but the changeover was a little awkward in a few places. Will AI be able to provide a quick solution to this problem too?

For the experiment, we copy the entire function that triggers the problem and the error message from the browser console into the prompt:

Prompt: JavaScript: When I call this function:

```
const _newComment = function (topic, meetingId, event) {
  event.target.disabled = true;
  fetch(`api/comments`, {
    body: JSON.stringify({
      id: topic,
      comment: this.newCommentContent.trim(),
      meetingId,
    }),
  })
  .then((json) => {
    this.comments.push({
      id: json.id,
```

```

        content: this.newCommentContent.trim(),
        date: moment().format("YYYY-MM-DD"),
        meeting: meetingId,
        user: this.$store("user").fullname,
    });
    this.newCommentContent = "";
  })
  .finally(() => {
    autosize.update(event.target);
    event.target.disabled = false;
    this.$focus.focus(event.target);
  });
};

```

I get this error message in my browser console window:

```

comments.js:8 Uncaught (in promise)
TypeError: Failed to execute 'fetch' on 'Window': Request with
  GET/HEAD method cannot have body.
    at Proxy._newComment (comments.js:8:3)
    ...

```

The error message you're seeing is due to the fact that you're not specifying the HTTP method in your fetch call, which defaults to GET. However, you're also trying to send a request body, which isn't allowed for GET requests. To fix this issue, you need to specify the HTTP method (likely POST) and include the necessary headers. Here's how you can modify your fetch call:

[response code]

Key changes:

1. Added method: 'POST' to specify that this is a POST request.
2. Added headers to specify the content type as JSON.
3. Changed `then(json => {` to `then(response => response.json()).then(json => {` to properly parse the JSON response.

Once again, the AI tool has impressed us: the error that the wrong HTTP method is used (GET instead of POST) when sending the data using fetch is almost obvious from the message in the browser console. But we didn't expect

two other errors (the missing content type and the missing conversion of the response from text to a JavaScript object with `response.json()`) to be resolved straight away.

3.1.5 JamGPT

In our search for tools with AI support for debugging, we came across the *Jam* browser extension. You can use this extension to easily document errors that you notice when using a web application. You can use it to create and share screenshots or screencasts (i.e., a video of the screen recording). In addition—and this is the great strength of the extension—a lot of additional information (e.g., the output of the browser console or the network requests executed) is recorded and also saved.

All information gets uploaded to the <https://jam.dev> platform, where each case, or *jam*, can be viewed and commented on. The service is freely available after registration, and paid access is available as expected for business and enterprise customers. Among other things, these offer the connection to an existing ticket system and login via the company network.

Now you may rightly ask what this has to do with AI. In Jam’s web interface, you’ll find the **JamGPT** tab on the far right of the screen, and this is where AI comes into play.

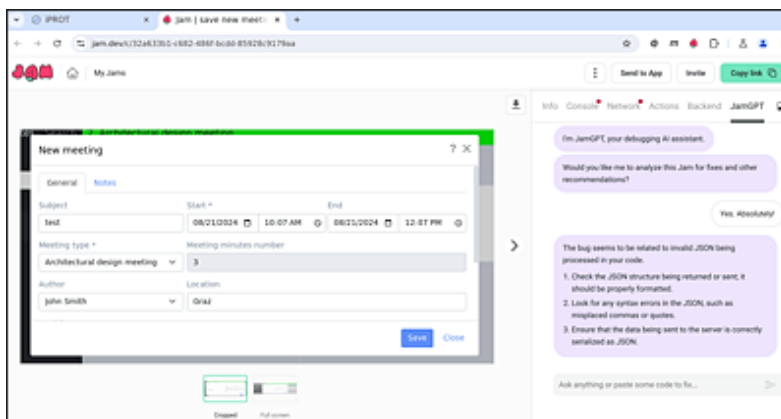


Figure 3.3 A Jam with AI Support for Faulty JavaScript Code

At the click of a button, you can send the error messages to a language model and hope for its solutions to the problem. The convenience of this solution is truly outstanding and demonstrates the possibilities, so we expected exceptional results. However, we were somewhat disappointed: for simple errors, such as a misspelled URL for a backend query, JamGPT correctly points out

the possible errors, but a web developer doesn't need AI support for this. The error code 404 is well known.

However, the AI tool was unable to shine in a more complex error that we produced for JamGPT. On the contrary, the answer even pointed in the wrong direction: to validate form data, we used a library that analyzes the `data-rules` HTML attribute of an element and then checks the validity of the rules stored there. The value of `data-rules` must be a character string that can be converted into a JavaScript object using `JSON.parse`, for example:

```
<input type="text" data-rules='["required"]'
name="email" />
```

Fortunately, with current browsers, you can do without this type of JavaScript-based validation, but the code is already several years old. The preceding character string is converted by `JSON.parse` into a JavaScript array with one entry, namely the `"required"` string. If you forget the double quotation marks around the `required` keyword, an error occurs in the JavaScript code of the library. The JavaScript console provides information about the point at which the conversion fails:

```
Uncaught SyntaxError: Unexpected token 'r', "[required]" is not
valid JSON
    at JSON.parse (<anonymous>)
    at validation.js:17:24
    ...
```

The response generated by JamGPT suggests that we check the server request or the server response for a valid JSON structure:

The bug seems to be related to invalid JSON being processed in your code.

1. Check the JSON structure being returned or sent; it should be properly formatted.
2. Look for any syntax errors in the JSON, such as misplaced commas or quotes.
3. Ensure that the data being sent to the server is correctly serialized as JSON.

But we hadn't even gotten that far yet: the error occurs before we can send a request to the server. Admittedly, most errors when converting JSON into web applications occur during the communication with a server, which is why the AI tool also suggests this as a possible problem. In our case, however, the problem wasn't related to the server, and the context in the browser console would have shown this. We're curious to see whether Jam will make any further progress in the integration of language models, as the potential undoubtedly exists.